

EE5332 Mapping DSP Algorithms to Architectures

Mini-Project

Gautam (EP23B013) & Bhavesh (EE23B016)

[Click here to access the codes](#)

1 Objectives

Resources are available here:

- Baseline HLS kernel `matmul_stream.cpp` (AXI-Stream I/O) for matrix-vector multiplication
- Two numerical variants: **Floating point** and **Fixed point** (`ap_fixed`)
- Unit tests that generate random matrices, vectors, and reference products
- Vitis HLS C simulation for both float and fixed
- HLS synthesis: report **resources** + **latency/II**
- Export RTL → integrate in Vivado → generate bitstream
- PYNQ notebook to load overlay, stream matrix + vectors via DMA, convert formats (float/fixed), verify correctness and measure timing

2 Challenges and Bugs Faced

1. **Fixed-mode streaming.** We spent a lot of time fixing this issue. Different conversion strategies were used for Python and the testbench to convert floating point into the required streaming format.
2. **DMA hang / wait never returns.** When the number of samples is large ($N_s = 1000$), the DMA hangs and data is not received while performing HW timing analysis. We suspect this is due to a counter overflow. We did not face this issue when the HW timer was disabled.
3. **Could not generate bitstream for $II = 1$.** It kept utilising 225 DSPs. Although the synthesis report says it needs only 192 DSPs, upon exporting we kept crossing 200 DSP usage. When implemented in Vivado it went above 225. It was an oversight as we forgot about the MMIO module which was part of the block design in Vivado.

However, we are still unable to fully explain why a few more DSPs are required when exporting the RTL.

3 Critical Design Choices

At the start we tried looping through the DMA by sending each iteration one by one, but we noticed that the DMA overhead was very high, so we decided to send all data together in a single DMA call.

We started with basic unoptimised multiply code. Since our end goal was pipelining, we first decided to eliminate the loop-carried dependency by changing the order of the loops and introducing a temporary variable.

```
// Matrix multiplication operation
matrix_mul:
for (int i = 0; i < MAT_SIZE; i++) {
    data_t temp = 0;
    ith_element:
    for (int j = 0; j < MAT_SIZE; j++) // calculate ith element
        temp += A[i][j] * X[j];
    Y[i] = temp;
}
```

We also considered tiling but deemed it unnecessary, as tiling is most beneficial for matrix–matrix multiplication and our entire architecture fit within the FPGA with no memory overhead.

Why AXI-Stream only instead of AXI-Lite + AXI-Stream? Unlike typical ML models, we do not continuously change the matrix elements. Configuring AXI-Lite and loading matrix data through it would add unnecessary delay.

4 Results

We ran the following solutions on the PYNQ board. All tests are for multiplication of a randomly generated 64×64 matrix with $N_s = 100$ samples of randomly generated 64×1 vectors.

4.1 ARM Processor Benchmark

Latency per sample:

Metric	Value
Mean	2.94×10^{-5} s (29.38 μ s)
Std Dev	3.52×10^{-5} s (35.17 μ s)
Median	1.18×10^{-5} s (11.78 μ s)
Min	1.14×10^{-5} s (11.42 μ s)
Max	1.06×10^{-4} s (106.34 μ s)

Multiplication was performed using NumPy and timing was measured with Python's `time` library.

4.2 Comparison Report

		Float	Fixed	Unroll	Array Part.	PL II=4	PL II=1
BRAM_18K		11	10	11	65	17	65
DSP		10	3	48	24	60	192
FF		9370	654	4300	4959	14640	23429
LUT		6389	719	2781	6342	8176	10408
URAM		0	0	0	0	0	0
ap_clk	Esti-	8.352 ns	6.912 ns	6.912 ns	6.912 ns	6.912 ns	6.912 ns
mated							
Latency/sample	(DMA)	87.8 μ s	374.0 μ s	107.0 μ s	83.3 μ s	73.0 μ s	59.8 μ s
Latency/sample	(HW Timer)	48.4 μ s	227.1 μ s	58.9 μ s	45.5 μ s	33.1 μ s	29.2 μ s
Avg L2 Error		0	3.8×10^{-6}	3.8×10^{-6}	3.8×10^{-6}	3.8×10^{-6}	3.8×10^{-6}

4.3 Floating Point — No Optimisation

One floating-point multiplication takes 5 DSP units; a fixed-point multiplication takes 3. This is expected as floating-point multiplication requires additional shifting.

- DMA: 10,496 input words, 6,400 output words in 8.78 ms $\Rightarrow$ 87.8 μ s/sample
- HW Timer: 484,009 cycles = 4,840.1 μ s @ 100 MHz $\Rightarrow$ Latency/sample = 48.4 μ s
- L2 error = 0

4.4 Fixed Point — No Optimisation

We used `ap_fixed<32, 26>` because 32 bits matches the AXI-Stream bus width. The unit tests were normalised with more fractional than integer values; 26 integer bits was found to be a sweet spot to minimise error. DSPs, FFs, and LUTs utilised are much fewer compared to floating point; however, floating point gives more accurate output.

- DMA: 10,496 input words, 6,400 output words in 37.40 ms $\Rightarrow$ 374.0 μ s/sample
- HW Timer: 2,270,822 cycles = 22,708.2 μ s @ 100 MHz $\Rightarrow$ Latency/sample = 227.1 μ s
- Average L2 error: 3.826×10^{-6}

4.5 Unrolled

The inner loop was unrolled by factor 16, so 16 multiplications are performed every cycle. DSP count: $16 \times 3 = 48$. Unrolling also requires more LUTs and FFs for addition. Latency decreases at the cost of more hardware units.

- DMA: 10,496 input words, 6,400 output words in 10.70 ms $\Rightarrow$ 107.0 μ s/sample
- HW Timer: 588,638 cycles = 5,886.4 μ s @ 100 MHz $\Rightarrow$ Latency/sample = 58.9 μ s
- Average L2 error: 3.820×10^{-6}

4.6 Array Partitioning

Cyclic BRAM partitioning with factor 32. Input vector **X** is partitioned with factor 32; matrix **A** is partitioned along columns with factor 32, since the inner loop accesses one row at a time. Increased BRAM usage is observed. Partitioning enables effective inner-loop unrolling.

- DMA: 10,496 input words, 6,400 output words in 8.33 ms $\Rightarrow$ 83.3 μ s/sample
- HW Timer: 455,413 cycles = 4,554.1 μ s @ 100 MHz $\Rightarrow$ Latency/sample = 45.5 μ s
- Average L2 error: 3.826×10^{-6}

4.7 Pipeline II = 4 and MUL Resources Constraint = 20

We pipelined the outer loop with II = 4 and attempted to unroll the inner loop by factor 16. However, Vitis completely unrolled the inner loop, ignoring the constraint. To prevent this, we set a resource constraint on multiplication. DSP count: $20 \times 3 = 60$.

- DMA: 10,496 input words, 6,400 output words in 7.30 ms $\Rightarrow$ 73.0 μ s/sample
- HW Timer: 330,892 cycles = 3,308.9 μ s @ 100 MHz $\Rightarrow$ Latency/sample = 33.1 μ s
- Average L2 error: 3.826×10^{-6}

4.8 Pipeline II = 1

The outer loop is pipelined with II = 1. The inner loop is automatically fully unrolled (by factor 64). Even though not specified in the directives, array partitioning is also performed to meet the requirements. A large number of hardware units are utilised.

- DMA: 10,496 input words, 6,400 output words in 5.98 ms $\Rightarrow$ 59.8 μ s/sample
- HW Timer: 292,291 cycles = 2,922.9 μ s @ 100 MHz $\Rightarrow$ Latency/sample = 29.2 μ s
- Average L2 error: 3.823×10^{-6}

5 Conclusion

Pareto-optimal curve for the design:

- All fixed-point solutions are Pareto-optimal with respect to hardware utilisation and latency.

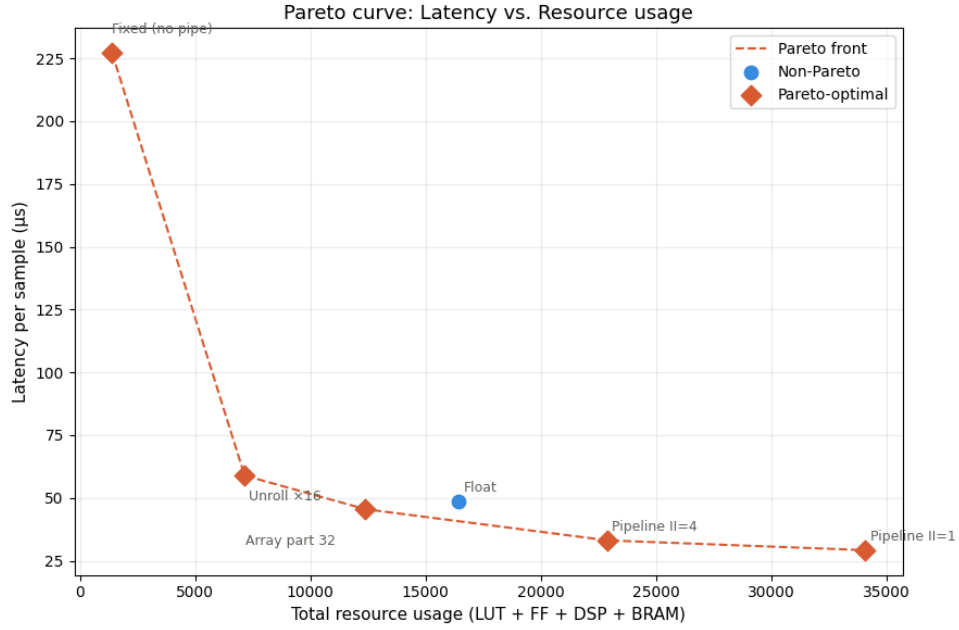


Figure 1: Pareto-optimal curve for the design

- Even though the floating-point solution is not Pareto-optimal on this curve, it is Pareto-optimal with respect to the error in the resultant vector.

6 Work Split

Note: Both of us did almost all the work together, actively finding problems in each other's approach.

Bhavesh S

- Wrote baseline HLS code for matrix multiplication along with the testbench.
- Experimented with various pragmas for optimal solutions.

Gautam D

- Helped with the testbench and converting it to Python for running on the Jupyter notebook.
- Integrated the HLS module with block design for DMA and AXI-Timer usage.
- Experimented with various pragmas for optimal solutions.